Student Marks Management API

Name: Ahmad Abdullah
Registration No.: 23f-0019
GitHub repository: https://github.com/ahmadchohan065-sudo/Students_Marks_Management

1. Introduction

This project is a Student Marks Management API for a university department. Teachers create students and courses, enroll students in courses and record marks. A student can see only his or her own record and transcript. It is built with C#, ASP.NET Core on .NET 10, Entity Framework Core and PostgreSQL. Users log in and receive a JWT, and the API is tested with a Postman collection and a built-in smoke test module.

2. Architecture

The code is split into layers so that each part has one job.

```
Client (Postman / Scalar / future website or app)
        |  HTTPS + JSON
        v
Endpoints   routes, authorization attributes, HTTP results
        |
        v
Services    business rules, logging (each behind an interface)
        |
        v
AppDbContext + Entities   Entity Framework Core
        |
        v
PostgreSQL  tables, keys, unique and check constraints
```

- **Endpoints** (`Endpoints/`) only choose the route and the required role, call a service and return the HTTP result. They contain no queries and no business rules.
- **Services** (`Services/`) hold the rules: the duplicate handling, the weighted average, the own-data rule for students, and the logging. Each service has an interface and is registered with dependency injection.
- **Data access** (`Data/`) is the scoped `AppDbContext`, the entities and the seeder.
- **DTOs** (`Dtos/`) are the only classes that cross the API boundary. Entities are never returned or accepted.
- **Common** (`Common/`) holds the custom exceptions, the global exception handler, the validation filter and the configuration option classes.
- **SmokeTests** (`SmokeTests/`) is a separate module that is started only on request.

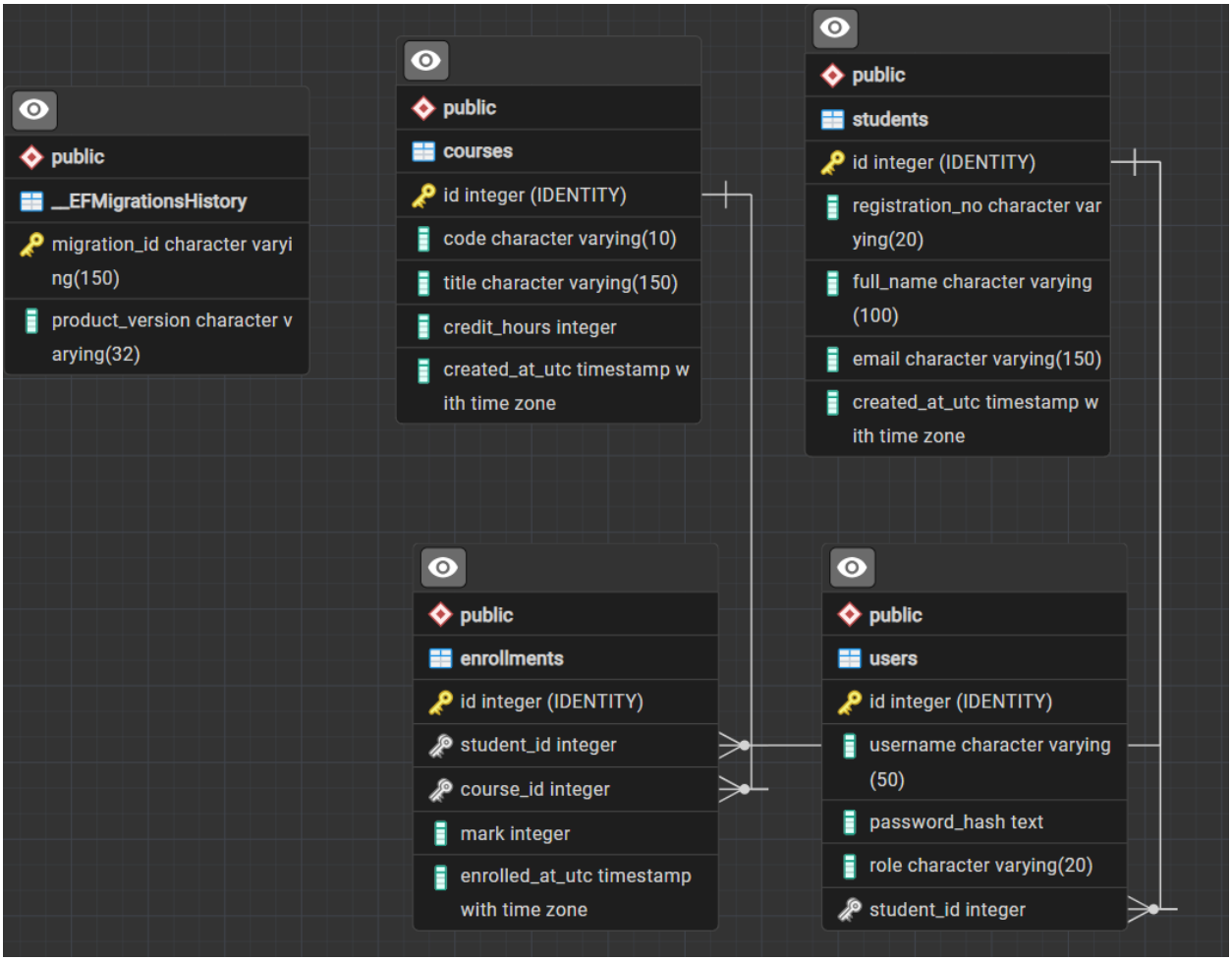
A typical request: the JWT is validated, the endpoint filter validates the request body, the endpoint calls the service, the service uses the DbContext, and the result goes back as a DTO. Any failure is thrown as an exception and turned into a problem-details response by the global handler.

3. Database design and ER diagram

The database has four tables with integer primary keys generated by the database.

- **students**: registration_no (unique), full_name, email (unique, stored in lower case), created_at_utc.
- **courses**: code (unique), title, credit_hours (check constraint, 1 to 4), created_at_utc.
- **enrollments**: student_id and course_id (both foreign keys with delete restricted), mark (null, or a check constraint from 0 to 100), enrolled_at_utc. A composite unique index on (student_id, course_id) allows a student to enroll in a course only once.
- **users**: username (unique), password_hash, role (Teacher or Student), and an optional student_id foreign key for users with the Student role.

One student has many enrollments, one course has many enrollments, and each enrollment links one student to one course and holds that student's mark. The tables and columns use snake_case names through the EFCore.NamingConventions package. The tables were created only with EF Core migrations, and the Migrations folder is committed. `db/schema.sql` is a schema-only export made with `pg_dump`.



4. Design decisions

Why 409 and not 400 for a duplicate. A duplicate request has a correct format, so it is not a bad request (400). The problem is that it conflicts with data that already exists, which is exactly what 409 Conflict means. A 400 is used when the request itself is invalid, for example an empty name or a mark of 101.

Why DTOs. Returning entities would expose internal fields such as the password hash and would tie the API shape to the database. Accepting entities would allow over-posting. DTOs let the API and the database change independently.

Why services behind interfaces. Endpoints depend on `IStudentService`, not on a concrete class. This keeps the endpoints thin and makes the services easy to replace or test.

Why delete is restricted. Foreign keys use `Restrict`, so a student or course that still has enrollments cannot be deleted. Marks cannot disappear by accident. The API returns 409 with a clear message.

Null for not graded. A mark of null means not graded yet, and 0 is a real mark. For this reason the course average is null (not 0) when nobody is graded, and `passed` is null for an ungraded course.

Weighted average. The transcript average is the sum of (mark x credit hours) divided by the sum of credit hours, over graded courses only. For example, CS401 with 3 credit hours and mark 80 and MA201 with 2 credit hours and mark 60 give (80 x 3 + 60 x 2) / 5 = 72. Averages are never averaged.

Configuration instead of constants. The pass mark (50), the JWT settings and the CORS origins are read from configuration. Secrets (connection string, JWT key) are stored in user-secrets in development and would be environment variables on a server. They are not in the repository. `appsettings.Example.json` shows every setting with fake values.

One error format. A global exception handler returns problem details for every error. Unexpected errors return a clean 500 with no stack trace, and the full exception goes only to the log.

Tokens and passwords. Passwords are stored only as hashes made by `PasswordHasher`. Login returns the same 401 message for an unknown user and a wrong password, so attackers cannot find out which usernames exist. Passwords and tokens are never logged.

Own-data rule. A student can read only his or her own record and transcript. The student id comes from the signed token, so it cannot be faked, and the check runs before the database lookup so the API does not reveal which ids exist.

Smaller decisions. The server corrects `page` and `pageSize` (maximum 50) and accepts only known `sort` values. All times use `DateTime.UtcNow` and `timestamptz` columns. All database calls are async and receive the `CancellationToken`. Read-only queries use `AsNoTracking` and project into DTOs with `Select`. Scalar (built on the .NET 10 OpenAPI support) is used for the API documentation.

5. How duplicates and concurrent requests are handled

A common mistake is to check whether a record exists and then insert it. Two requests can arrive at the same moment, both pass the check, and both insert. This project does not check first.

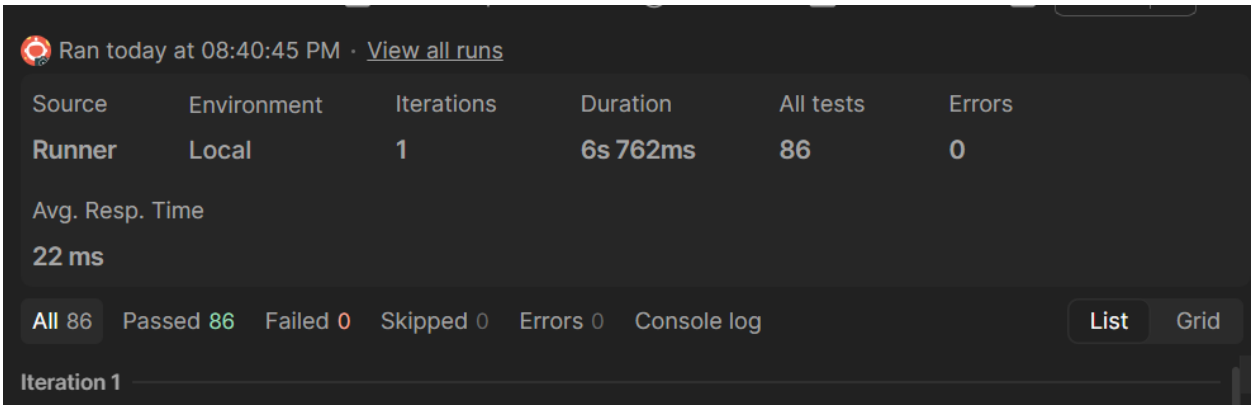
1. The database has unique indexes on registration number, email, course code, username, and on (student_id, course_id) for enrollments.
2. The service inserts straight away and calls `SaveChangesAsync`.
3. If the index is violated, PostgreSQL raises error code 23505. The service catches the `DbUpdateException`, recognises this code and throws a conflict exception.
4. The global handler turns it into a 409 problem-details response.

When two requests race, the database lets exactly one succeed and the other receives 409. The same idea is used for delete: if a foreign key blocks the delete (error 23503), the service returns 409. The `DbContext` is registered as scoped, so every request gets its own instance, and there is no static mutable state.

6. Testing

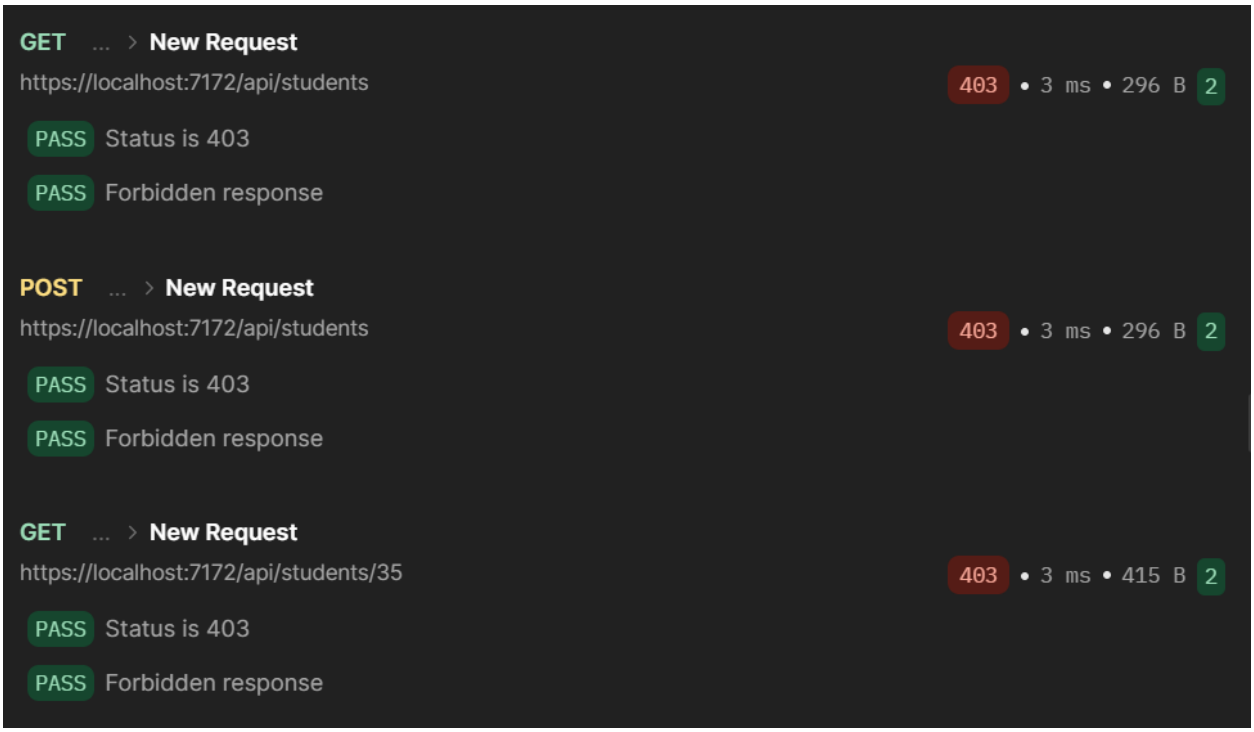
Postman

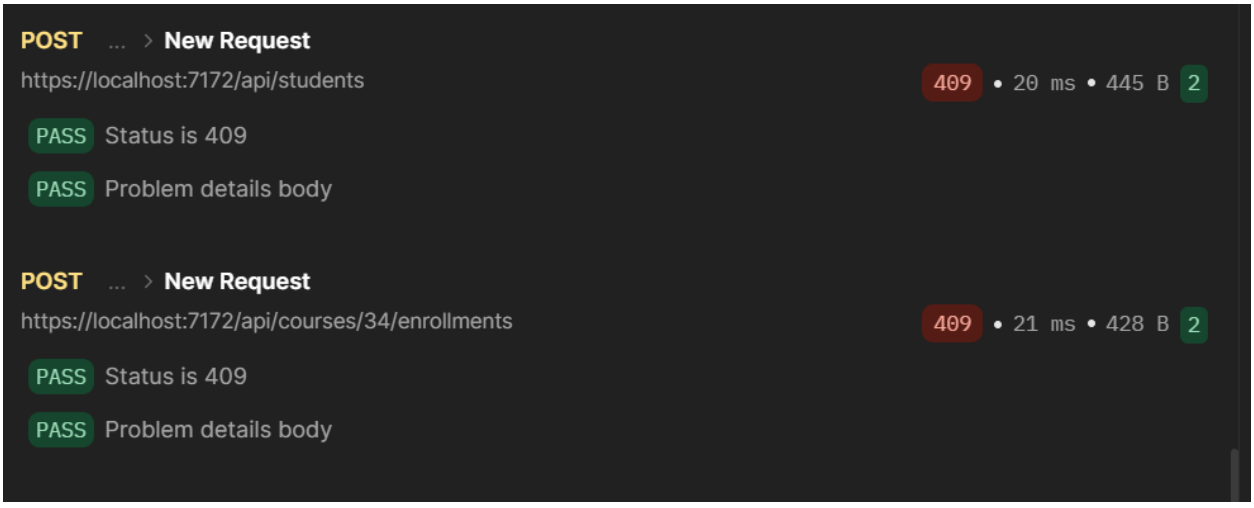
The collection **Marks API** has 43 requests in these folders, in this run order: Auth, Students, Courses, Enrollments and Marks, Reports, Health, Negative Tests. The collection uses Bearer authorization with `{{token}}`, the login request saves the token automatically, and create requests save ids in variables that later requests use. Every request has at least two tests, and there are 40 negative tests covering 400, 401, 403, 404 and 409. The collection is run on a fresh database because it creates records with fixed codes.





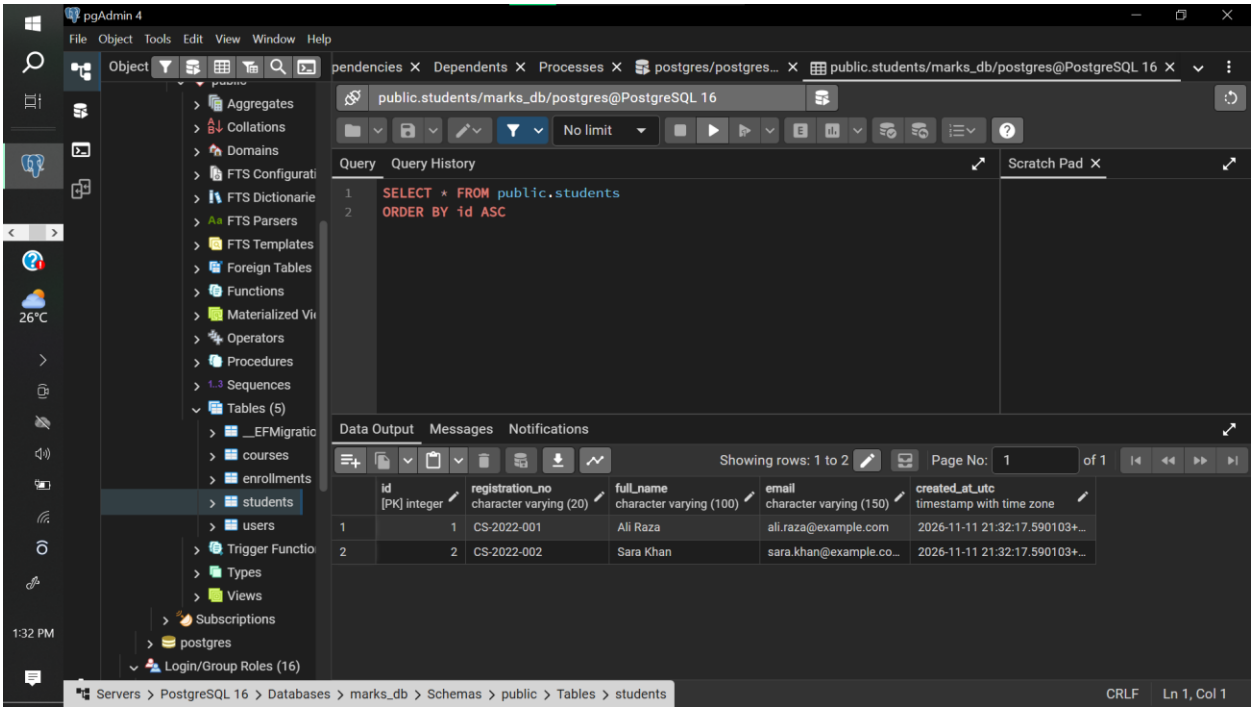

POST ... > New Request
https://localhost:7172/api/auth/login
400 • 2 ms • 413 B 2
PASS Status is 400
PASS Problem details body


GET ... > New Request
https://localhost:7172/api/students
403 • 3 ms • 296 B 2
PASS Status is 403
PASS Forbidden response
POST ... > New Request
https://localhost:7172/api/students
403 • 3 ms • 296 B 2
PASS Status is 403
PASS Forbidden response
GET ... > New Request
https://localhost:7172/api/students/35
403 • 3 ms • 415 B 2
PASS Status is 403
PASS Forbidden response


POST ... > New Request
https://localhost:7172/api/students
409 • 20 ms • 445 B 2
PASS Status is 409
PASS Problem details body
POST ... > New Request
https://localhost:7172/api/courses/34/enrollments
409 • 21 ms • 428 B 2
PASS Status is 409
PASS Problem details body

Smoke test

The smoke test is started with `dotnet run --launch-profile https -- --smoke-test`. It runs only on request, sends real HTTP requests to the running API and the real PostgreSQL database, reads the login details from configuration, and deletes all the data it created (prefix SMOKE- or SMK). It prints one PASS or FAIL line per check, a summary line, and exits with code 0 only if all checks pass.



7. Problems faced and how I solved them

1. `psql` was not recognised. The command failed because the PostgreSQL `bin` folder was not on the Windows PATH. I created the user and the database from the Query Tool in pgAdmin instead. Adding the folder to PATH would also work.

2. Old .NET version and Swagger. The guide I started from assumed .NET 8, but only .NET 10 was installed and supported on my computer. I created the project on .NET 10 and installed package versions that match it. The newest Swashbuckle (Swagger) uses a different OpenAPI library, so I used the built-in .NET 10 OpenAPI support with Scalar. The assignment allows either tool. I first opened `/swagger` and got a 404, then realised the documentation page is now `/scalar/v1`.

3. Build errors from an outdated Program.cs. The first build failed because `Program.cs` still had the old `Microsoft.OpenApi.Models` using line and referred to the smoke test module before I had created it. I replaced the Swagger part and created the `SmokeTests` folder and file.

4. Forgotten database password. I recovered it from `dotnet user-secrets list`, since the connection string is stored there. I also learned to supply it to `pg_dump` through the `PGPASSWORD` variable for one command, so it is never written into a file or into Git.

5. Postman: empty `baseUrl`. Every request uses `{{baseUrl}}`, but the Local environment had no value, so no request worked. I set `baseUrl` to `https://localhost:7172`.

6. Postman: wrong folder order. The collection ran Courses last, so the enrollment requests used an empty `{{courseId}}` and returned 404. I moved Courses to run before Enrollments: Auth, Students, Courses, Enrollments, Reports, Health, Negative Tests.

7. Postman: leftover data causing 409. Create requests used fixed codes such as `PM-001` and `PM101` that already existed from earlier runs. I changed the creation requests and the update requests to fresh values (`PM-401`, `PM401`) and noted that the collection must run on a fresh database.

8. Postman: negative tests out of date. The duplicate tests still used the old codes, so they created records (201) instead of conflicting (409). I updated them to reuse the new values. The duplicate student test also has to use values that still exist after the update request changes the student's email, so it uses `updatedpm401@example.com`. Both the registration number and the email are unique, so reusing either one gives 409.

9. Postman: wrong assumptions in tests. The test for `GET /api/courses` expected a plain array, but the endpoint returns a paged object with `items`, `page`, `pageSize` and `totalCount`, so I changed the test to check those keys. The test for `PUT /api/courses` still checked the old title after I changed the request body, so I updated the assertion.

8. Conclusion

The API meets the requirements of the assignment: a PostgreSQL schema built with migrations, endpoints with correct status codes and one standard error format, JWT authentication with role rules and the student own-data rule, logging, a health check, CORS limited to configured origins, API documentation, a Postman collection, and a smoke test.